

Проектирование поиска инструкций

Цель

Сделать для Telegram и MAX функцию Поиск инструкции, которая позволит пользователю ввести:

- название актива
- синоним
- INCI

и получить одну или несколько ссылок на подходящие посты с инструкциями из истории каналов.

Эта функция отделена от будущего Каталога рецептов.

Посты-рецепты, где используются несколько активов, будут позже реализованы как отдельный продуктовый сценарий.

Пользовательский сценарий

Как это работает для пользователя

- Пользователь нажимает Поиск инструкции.
- Бот переходит в режим ожидания запроса и пишет:

Введите название актива, синоним или INCI.

- Пользователь отправляет запрос.
- Бот нормализует текст.
- Бот ищет совпадения в индексе инструкций.
- Бот отвечает:
- если найден один точный результат: отправляет название + короткое описание + ссылку
- если найдено несколько сильных совпадений: отправляет 3-5 лучших вариантов
- если ничего не найдено: предлагает переформулировать запрос или обратиться к менеджеру

Как должен работать поиск

Поиск должен срабатывать по:

- русскому названию актива
- синонимам
- INCI
- нормализованным вариантам без регистра, лишних пробелов и знаков

Примеры:

- ниацинамид
- витамин b3
- niacinamide
- azelaic acid

Модель данных

Рекомендуемая структура хранения

Используем ту же Google-таблицу, которую уже создали, но разделяем её на специализированные листы.

Рекомендуемые вкладки:

- ANALYTICS_MAX

Текущая аналитика и события MAX-бота.

- ANALYTICS_TELEGRAM

Текущая аналитика и события Telegram-бота.

- POSTS_TELEGRAM

Сырые и структурированные посты-инструкции из Telegram-каналов.

- POSTS_MAX

Сырые и структурированные посты-инструкции из MAX-каналов.

- INSTRUCTION_INDEX

Единый поисковый индекс, из которого работают боты.

- SYNONYMS

Таблица синонимов, алиасов и соответствий INCI.

- SYNC_STATE

Техническое состояние синхронизации и служебные метки.

- RECIPE_BACKLOG

Отдельный parking lot для будущего каталога рецептов и постов с несколькими активами.

POSTS_TELEGRAM и POSTS_MAX

Каждая строка — один исходный пост.

Рекомендуемые колонки:

- source_platform
- source_channel_name
- source_channel_id
- source_post_id
- post_url
- published_at_utc
- title_raw
- text_raw
- media_flag
- is_instruction
- is_recipe_candidate
- extracted_active_primary
- extracted_active_secondary
- extracted_inci_primary
- tags_json
- parse_status
- last_synced_at_utc

INSTRUCTION_INDEX

Это быстрый и удобный слой для поиска ботом.

Рекомендуемые колонки:

- instruction_id
- source_platform
- source_channel_name
- source_post_id
- post_url
- display_title
- active_name
- active_name_normalized
- synonyms_json
- inci_json
- search_text
- rank_weight
- status
- updated_at_utc

SYNONYMS

Один и тот же актив может иметь несколько форм записи.

Рекомендуемые колонки:

- canonical_active
- canonical_active_normalized
- synonym
- synonym_normalized
- inci
- inci_normalized
- language
- notes

Пример:

- Ниацинамид
- Витамин B3
- Niacinamide

Как будет работать загрузка данных

Этап 1: сбор истории

Да, историю постов можно и нужно выгружать и структурировать.

Предпочтительный процесс:

- Выгружаем или получаем историю постов из Telegram-каналов.
- Выгружаем или получаем историю постов из MAX-каналов.
- Сохраняем сырой текст поста и метаданные в POSTS_TELEGRAM / POSTS_MAX.
- Прогоняем parser/enricher и строим INSTRUCTION_INDEX.

Этап 2: инкрементальные обновления

Автоматическое обновление:

- базовый вариант: 1 раз в день
- возможное усиление позже: каждые 6 часов
- 1 раз в неделю для клиентской функции слишком редко, если контент обновляется активно

Рекомендуемый старт:

- ежедневная синхронизация Telegram и MAX
- отдельная ручная команда для срочного переиндекса

Логика парсинга

Для каждого поста:

- определить, является ли пост инструкцией
- извлечь кандидатов на название актива
- сопоставить найденные формы с таблицей SYNONYMS
- обогатить запись каноническим названием и INCI
- записать результат в INSTRUCTION_INDEX

Посты с несколькими активами

Для этой функции:

- если пост в основном посвящён одному активу, индексируем его как инструкцию
- если это полноценный пост-рецепт с несколькими активами, ставим:
- `is_recipe_candidate = true`
- не смешиваем его в v1 с поиском инструкций, если это не нужно явно

Это и есть тот самый слой "будущего каталога рецептов", к которому нужно будет вернуться позже.

Поведение бота

Изменения в интерфейсе

В обоих ботах должна появиться кнопка Поиск инструкции:

- в основном меню после авторизации
- в закреплённой нижней клавиатуре/панели ввода там, где это поддерживается

Машина состояний

Нужно простое состояние пользователя:

- `idle`
- `awaiting_instruction_query`

Когда пользователь нажимает Поиск инструкции:

- состояние меняется на `awaiting_instruction_query`

- следующее текстовое сообщение трактуется как поисковый запрос

После выдачи результата:

- возвращаем пользователя в idle

Конвейер поиска

Когда приходит запрос:

- переводим в нижний регистр
- убираем лишние пробелы
- нормализуем знаки и пунктуацию
- позже при необходимости добавляем более умную нормализацию и транслитерацию
- ищем точные совпадения по:
 - active_name_normalized
 - synonym_normalized
 - inci_normalized
- если точных совпадений нет, используем частичное совпадение по search_text
- сортируем результаты:
 - сначала точное совпадение с каноническим названием
 - потом точное совпадение по синониму или INCI
 - затем частичные совпадения

Формат ответа

Если результат один:

- Нашёл инструкцию: Ниацинамид
- ссылка на пост

Если результатов несколько:

- Нашёл несколько вариантов:
- список из 3-5 названий со ссылками

Если результатов нет:

- Не нашёл инструкцию. Попробуйте другое название, синоним или INCI. Или свяжитесь с менеджером
- кнопка связи с менеджером

Сильные стороны подхода

- Быстро запускается
- Поиск получается детерминированным и проверяемым
- Ошибки легко правятся вручную через таблицу
- Работает сразу для двух мессенджеров через единый индекс
- С первого дня поддерживает поиск по названию, синонимам и INCI

Слабые стороны подхода

- Нужно один раз построить пайплайн загрузки истории
- Автоматический разбор сырого контента может быть шумным, если посты оформлены по-разному
- Google Sheets удобен как управляющий слой, но не идеален как долгосрочный high-scale backend для поиска
- По MAX выгрузка может быть слабее/сложнее, чем по Telegram, в зависимости от доступности API и фактических возможностей платформы

Рекомендуемая реализация v1

Стек

- Боты продолжают работать как сейчас
- Скрипт синхронизации и индексации работает на VPS
- Google Sheets остаётся управляющим слоем и визуальной админкой
- Локальный SQLite на VPS используется как быстрый runtime-кэш для поиска

Лучшая практическая архитектура

- Сырые посты пишем в листы таблицы
- Единый индекс тоже сохраняем в таблицу для прозрачности
- Бот периодически зеркалит INSTRUCTION_INDEX в локальный SQLite
- Поисковые запросы пользователей обрабатываются уже по локальному SQLite, а не по live Google Sheets

Это даёт:

- удобное ручное управление через Sheets
- быстрые ответы бота из локальной базы

Как усилить систему позже

Усиление 1: SQLite или Postgres как основной поисковый индекс

Когда каталог вырастет:

- Sheets остаётся административным и обзорным слоем
- база данных становится runtime source of truth

Усиление 2: fuzzy search

Добавляем устойчивость к опечаткам:

- trigram similarity
- Levenshtein-подобный поиск
- нормализация транслитерации

Усиление 3: семантический слой через дешёвый или бесплатный LLM backend

Да, систему можно усилить AI-слоем.

Лучший малозатратный путь:

- точный поиск по названию/синонимам/INCI остаётся основным
- LLM используется как fallback reranker или как фоновый extractor

Практичные варианты:

- ollama с локальными embeddings/rerank, если хватит ресурсов VPS
- дешёвые hosted-модели через Groq/OpenRouter только для этапа индексации
- open-source NER/extraction pipeline для автоматического выделения активов

Рекомендуемое применение LLM:

- извлекать названия активов из сырых постов
- классифицировать пост как инструкция или рецепт
- генерировать короткий нормализованный заголовок

Что не рекомендуется для v1:

- отправлять каждый live-запрос пользователя в LLM

Почему:

- медленнее
- дороже
- менее предсказуемо, чем строгий поиск по индексу

Этапы внедрения

Фаза А

- добавить кнопку Поиск инструкции в оба бота
- добавить пользовательское состояние awaiting_instruction_query
- создать нужные листы в таблице

Фаза В

- построить загрузку исторических постов из Telegram
- построить загрузку исторических постов из MAX
- записать сырые посты в POSTS_*

Фаза С

- сделать parser/indexer
- заполнить INSTRUCTION_INDEX
- заполнить SYNONYMS

Фаза D

- подключить поиск в боте
- запустить exact + synonym + INCI поиск
- оформить выдачу результатов

Фаза Е

- настроить ежедневную синхронизацию
- добавить ручную переиндексацию
- усилить ranking/fuzzy search

Зафиксированные решения

- Источник данных: посты из каналов Telegram и MAX
- Один пост = одна инструкция для этой функции
- Посты-рецепты с несколькими активами откладываются в отдельный будущий каталог
- Поля поиска: название + синоним + INCI
- Загрузка: автоматическая
- Частота обновления: рекомендуется ежедневно
- Отдельные листы для постов по платформам: да